

LUT-Based Partial Computation Replacement

Practical Improvement and Hardware Deployment Plan

1. Current Status

We have completed a hardware-level decomposition benchmark for the LUT-based partial replacement of the `down_proj` layer in `Qwen/Qwen2.5-7B-Instruct`.

The current configuration is:

- Model: Qwen2.5-7B-Instruct
- Tested layer: Layer 21
- Hidden size: 3584
- Intermediate size: 18944
- Sequence length: 128
- Batch size: 1
- Number of replaced groups: 16
- Channels per group: 64
- Total replaced output channels: 1024
- Replacement ratio: 28.57%
- LUT storage: approximately 16 MB in FP32

The most important result is that the v3 implementation is able to physically skip part of the original dense matrix multiplication.

The original full `down_proj` matmul takes:

0.1679 ms

After replacing 1024 output channels, the active partial matmul takes:

0.1270 ms

This means that reducing the output dimension by 28.57% produces an actual matmul latency reduction of approximately 24.4%.

Therefore, the partial computation skipping mechanism itself is valid.

However, the current GPU implementation also introduces:

Triton LUT generation: 0.1137 ms
Output reconstruction: 0.0174 ms

The final GPU latency becomes:

0.2581 ms

which is slower than the original dense matmul.

This does not mean that the replacement mechanism is ineffective. It means that the current execution structure is not suitable for commodity GPU deployment, because GPU-optimized dense GEMM is extremely efficient, while separate LUT lookup, kernel launch, intermediate tensor generation, and output reconstruction create additional overhead.

The following development plan therefore does not target GPU latency improvement. Its goal is to determine whether the method can produce meaningful computation, memory-access, energy, and latency savings on hardware designed to support LUT and matrix computation jointly.

2. Main Practical Question

The project should no longer focus on the question:

Can the current implementation outperform cuBLAS or PyTorch dense GEMM on a general-purpose GPU?

The practical question should instead be:

When LUT lookup and partial matrix multiplication are supported by a specialized hardware dataflow, how much full-model computation, memory access, energy consumption, and inference time can actually be reduced?

This question must be answered at the full-model level.

A single-layer reduction is not sufficient to justify customized hardware.

3. Current Full-Model Contribution

The current experiment replaces 28.57% of one `down_proj` layer.

For one Transformer block, `down_proj` is only one part of the total linear computation. The same block also contains:

- Query projection
- Key projection
- Value projection
- Attention output projection
- `gate_proj`
- `up_proj`
- `down_proj`

Using the Qwen2.5-7B dimensions, the `down_proj` operation represents approximately 26.6% of the major linear computation in one Transformer block.

Replacing 28.57% of `down_proj` therefore reduces the major linear computation of that block by approximately:

$$26.6\% \times 28.57\% \approx 7.6\%$$

If only one layer is replaced in a 28-layer model, the ideal full-model computation reduction is only approximately:

$$7.6\% / 28 \approx 0.27\%$$

Therefore, the current single-layer result is technically valid but has limited product-level impact.

Approximate ideal full-model reductions under the same replacement ratio are:

Number of replaced layers	Ideal reduction of major model linear computation
1	0.27%
4	1.1%
8	2.2%
16	4.3%
28	7.6%

These numbers represent upper-bound computation reductions under the current design, before considering non-matrix operations, memory overhead, control overhead, and hardware utilization.

This leads to the first major conclusion:

The next priority is not further optimization of a single layer. The next priority is to determine whether the replacement can be expanded to enough layers and channels to produce a meaningful full-model reduction.

4. Core Deployment Metric

The main system-level metric should be changed from the number of replaced groups to the full-model replaced MAC ratio:

$$R_{\text{model}} = \frac{\text{Total dense MACs eliminated by LUT replacement}}{\text{Total model MACs}}$$

For every configuration, the following metrics should be reported together:

- Full-model replaced MAC ratio
- Number of replaced layers
- Replacement ratio per layer
- LUT precision
- Total LUT storage
- LUT read traffic per token
- Accuracy or task-performance degradation
- Logit or hidden-state divergence
- Estimated hardware cycle reduction
- Estimated energy reduction

The group count is an implementation parameter. It should not be treated as the final business or deployment metric.

5. Practical Go/No-Go Thresholds

Customized hardware should not be considered merely because a single layer can skip part of its matrix multiplication.

A practical threshold should be set based on full-model benefit.

5.1 Below 3% Full-Model Computation Reduction

This is unlikely to justify customized hardware.

After accounting for:

- Attention operations
- KV cache access
- Normalization
- Sampling
- Memory movement
- Control logic
- CPU orchestration
- Non-replaced layers

a theoretical 3% MAC reduction may become approximately 1% or less at system level.

The engineering cost is unlikely to be justified.

5.2 Between 5% and 10%

This range may justify:

- FPGA prototyping
- NPU extension

- Specialized accelerator evaluation
- Hardware-software co-design

The value may be meaningful in energy-sensitive or throughput-sensitive deployment.

5.3 Between 10% and 20%

This range has clear product value, especially for:

- Edge inference
- Vehicle-mounted inference
- Robotics
- On-device language models
- High-throughput inference servers
- Power-constrained deployment

At this point, dedicated hardware support becomes worth serious evaluation.

5.4 Above 20%

This level could justify:

- Dedicated ASIC logic
- CIM mapping
- Specialized SRAM-LUT structures
- A custom NPU dataflow

However, reaching this range may require extending replacement beyond `down_proj` to other linear layers.

6. Immediate Improvement Direction

6.1 Expand from One Layer to Multiple Layers

The next experiment should cover multiple middle and later Transformer layers.

Different layers should not be forced to use the same replacement ratio.

Each layer should be tested independently with:

4 groups
8 groups
12 groups
16 groups

For each layer and group setting, record:

- Replaced output channels
- Replaced MACs
- Local KL divergence
- Output error
- Perplexity change
- Task-performance change
- LUT storage
- LUT bandwidth requirement

The expected result is a non-uniform layer configuration.

For example:

Early layers: 0–4 groups
Middle layers: 8–12 groups
Late-middle layers: 12–16 groups
Final layers: 4–12 groups

The final deployment configuration should maximize total eliminated MACs while keeping model quality within the acceptable range.

6.2 Evaluate Multi-Layer Accumulated Error

Single-layer KL is not sufficient.

When multiple layers are replaced, approximation errors may accumulate through the model.

The following combinations should be tested:

Configuration	Description
4-layer conservative	4 middle or late layers, 8 groups each
8-layer conservative	8 layers, 8 groups each
8-layer aggressive	8 layers, 16 groups each
16-layer conservative	16 layers, 8 groups each
Adaptive configuration	Different group count for each layer

The adaptive configuration is the most important because layer sensitivity is unlikely to be uniform.

7. Accuracy and Product-Quality Evaluation

The current KL improvement is useful, but local KL alone cannot determine whether the model remains usable.

The evaluation should be divided into three levels.

7.1 Token-Level Metrics

- Logit KL divergence
- Top-1 token agreement
- Top-5 token overlap
- Perplexity
- Token-level cross entropy

7.2 Task-Level Metrics

The actual task suite should reflect the intended product scenario.

Possible categories include:

- Question answering
- Instruction following
- Summarization
- Structured output
- Coding
- Classification
- Multi-turn conversation
- Domain-specific generation

7.3 Long-Generation Stability

Approximation error may accumulate during autoregressive generation.

At minimum, evaluate:

32 generated tokens
128 generated tokens
512 generated tokens
Long-context generation

A configuration should not be accepted merely because the first-token or single-layer KL is low.

The practical acceptance criterion should be:

The full-model computation reduction reaches at least 5%–10%, while the target-task quality degradation remains within the product's acceptable range.

8. LUT Storage Optimization

The current LUT uses FP32.

For 16 groups:

One layer: 16 MB

If applied to multiple layers:

Number of layers	FP32 LUT storage
1	16 MB
8	128 MB
16	256 MB
28	448 MB

This is acceptable in GPU memory but may be too large for local SRAM on customized hardware.

LUT quantization should therefore be developed in parallel with multi-layer replacement.

8.1 Precision Targets

LUT precision	Storage per 16-group layer
FP32	16 MB
FP16/BF16	8 MB
INT8	4 MB
INT4	2 MB

The first practical target should be FP16 and INT8.

INT4 can be explored later if INT8 remains stable.

8.2 Required Experiments

For each LUT precision, evaluate:

- Hidden-state KL
- Logit KL
- Perplexity
- Task accuracy
- Long-generation stability

- Total storage
- Read bandwidth
- Quantization overhead
- Scale and zero-point storage
- Per-group versus per-channel quantization

8.3 Further Compression Options

If LUT storage remains too large, evaluate:

- Shared codebooks
- Cross-layer table sharing
- Low-rank LUT output
- Product quantization
- Sparse LUT representation
- Vector quantization
- Table pruning
- Entry clustering
- Delta encoding

The objective is not only to reduce capacity but also to reduce SRAM access energy and bandwidth.

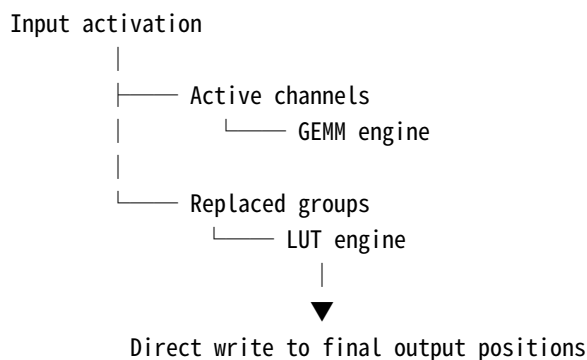
9. Hardware-Friendly Execution Design

The current GPU execution sequence is:

```
Partial matmul
→ LUT generation
→ Create full output
→ index_copy / reconstruction
```

This structure should not be copied into customized hardware.

The target hardware dataflow should be:



The hardware design should satisfy the following conditions:

- Active GEMM and LUT generation execute in parallel
- LUT data is stored close to the compute unit
- LUT results directly write to final output addresses
- No separate full-output reconstruction step
- No `index_copy`
- No additional intermediate full-size output tensor
- Replaced groups align with hardware tiles
- LUT channel locations are statically configured
- Output write conflicts are avoided
- LUT read latency is hidden behind GEMM execution

The ideal layer latency model is:

$$T_{\text{layer}} = \max(T_{\text{active GEMM}}, T_{\text{LUT}}) + T_{\text{output conflict}}$$

If LUT processing is faster than the remaining GEMM and output conflicts are negligible:

$$T_{\text{layer}} \approx T_{\text{active GEMM}}$$

Under this condition, the removed output tiles can translate into real latency reduction.

10. Hardware Tile Alignment

The current group size is 64 channels.

This may be useful if the target hardware uses arrays or output tiles with widths such as:

64
128
256

The hardware cycle reduction depends on the number of eliminated tiles, not only on the mathematical reduction in output channels.

For an output array width N_{PE} :

$$N_{\text{tiles}} = \left\lceil \frac{N_{\text{active}}}{N_{\text{PE}}} \right\rceil$$

The baseline and replacement configurations should be compared using actual tile counts.

For example, if a hardware engine processes 64 output channels per tile:

Baseline output channels: 3584
Baseline tiles: 56

Active output channels: 2560
Active tiles: 40

This removes 16 complete output tiles.

That is a meaningful hardware improvement.

However, if the group arrangement leaves fragmented active channels or fails to reduce the tile count, the theoretical channel reduction may not translate into latency reduction.

Therefore, group selection should become hardware-aware.

The selection objective should consider both:

- Model approximation quality
- Hardware tile eliminability

11. Cycle Model

Before accessing real customized hardware, build a parameterized cycle model.

The model should include:

- GEMM array width
- GEMM array height
- MACs per cycle
- Number of active output tiles
- Input tile count
- Reduction depth
- LUT reads per cycle
- LUT entry width
- LUT SRAM bandwidth
- LUT address-generation cycles
- Output write bandwidth
- GEMM and LUT concurrency
- Buffer capacity
- Pipeline startup and drain cost

The baseline model is:

$$T_{\text{baseline}} = T_{\text{full GEMM}}$$

The replacement model is:

$$T_{\text{replacement}} = \max(T_{\text{partial GEMM}}, T_{\text{LUT path}}) + T_{\text{write conflict}} + T_{\text{pipeline}}$$

The cycle model should generate results for multiple target hardware assumptions:

- 64-column engine
- 128-column engine
- 256-column engine
- Systolic array
- SRAM-based digital CIM
- FPGA MAC array

The output should show:

- Layer cycle reduction
- Full-model cycle reduction
- LUT bandwidth requirement
- Required SRAM capacity
- Hardware utilization
- Bottleneck location

12. Energy Model

Energy savings should not be assumed to equal the replacement ratio.

The baseline energy model should include:

$$E_{\text{baseline}} = N_{\text{MAC}} E_{\text{MAC}} + N_{\text{weight-read}} E_{\text{weight-read}} + N_{\text{activation-read}} E_{\text{activation-read}} + N_{\text{output-write}} E_{\text{output-write}} + E_{\text{control}}$$

The replacement energy model should include:

$$E_{\text{replacement}} = N_{\text{active-MAC}} E_{\text{MAC}} + N_{\text{active-weight-read}} E_{\text{weight-read}} + N_{\text{LUT-read}} E_{\text{LUT-read}} + N_{\text{address}} E_{\text{address}} + N_{\text{output-write}}$$

The analysis should explicitly include:

- MAC energy eliminated
- Dense weight-access energy eliminated
- LUT SRAM read energy added
- Address-generation energy added
- LUT static leakage
- Buffer and output-write energy
- Data movement between compute and memory

The most important practical output is the break-even condition:

Under what LUT read energy, SRAM placement, and hardware bandwidth does the replacement consume less energy than the original dense computation?

13. Suggested Implementation Phases

Phase 1: Multi-Layer Software Validation

Objective

Determine whether the method can reduce full-model computation by at least 5%–10% while maintaining acceptable task quality.

Tasks

- Select 8–16 middle and later layers
- Test 4, 8, 12, and 16 groups per layer
- Build an adaptive per-layer configuration
- Add FP16 LUT
- Add INT8 LUT
- Measure multi-layer accumulated error
- Run real task benchmarks
- Calculate full-model replaced MAC ratio
- Calculate total LUT capacity
- Calculate LUT traffic per token

Deliverables

- Layer sensitivity map
 - Multi-layer Pareto curve
 - Recommended deployment configuration
 - Full-model MAC reduction
 - Total LUT storage
 - Accuracy degradation report
 - Go/no-go recommendation for hardware prototyping
-

Phase 2: Hardware Mapping and Analytical Modeling

Objective

Estimate whether realistic customized hardware can translate the eliminated MACs into real latency and energy savings.

Tasks

- Define target hardware abstractions
- Build tile-level cycle model
- Build memory-bandwidth model
- Build LUT SRAM model
- Build energy model
- Model direct output placement
- Model GEMM/LUT parallel execution
- Estimate full-model latency reduction

- Estimate full-model energy reduction

Deliverables

- Hardware dataflow diagram
 - Cycle model
 - Energy model
 - Bandwidth requirements
 - SRAM capacity requirements
 - Break-even analysis
 - Recommended FPGA or accelerator configuration
-

Phase 3: FPGA or Programmable Accelerator Prototype

Objective

Validate that the LUT path can operate in parallel with active GEMM and remain outside the critical path.

Prototype Scope

A full 7B model is not required initially.

The first prototype can implement:

- One realistic `down_proj` layer
- Active-channel GEMM path
- LUT address generator
- LUT SRAM
- Direct output placement
- Parallel GEMM/LUT scheduling
- Cycle counters
- Power measurement if available

Questions to Answer

- Can the LUT path be hidden behind GEMM?
- Is SRAM bandwidth sufficient?
- Does address generation become a bottleneck?
- Are output writes conflicting?
- Can the target clock frequency be maintained?
- What is the measured energy reduction?
- What is the real latency reduction?
- How much hardware area does the LUT engine require?

Deliverables

- FPGA or RTL prototype
- Measured cycle reduction
- Measured power or energy reduction
- Resource utilization

- Maximum clock frequency
 - Bottleneck report
 - Recommendation for further hardware investment
-

Phase 4: Customized Hardware Evaluation

This phase should only begin if all previous conditions are satisfied.

Possible directions include:

- Custom NPU extension
- ASIC LUT unit
- SRAM-based CIM
- Digital CIM
- Dedicated inference accelerator
- Hardware-software co-designed edge processor

The decision should be based on measured or strongly modeled product-level benefits, not on single-layer microbenchmarks.

14. Go/No-Go Criteria for Customized Hardware

The project should proceed to hardware prototyping only if most of the following conditions are satisfied:

1. Full-model MAC reduction reaches at least 5%, preferably 10% or more.
2. Target-task quality degradation remains within product tolerance.
3. FP16 or INT8 LUT remains stable.
4. Total LUT storage fits the expected memory hierarchy.
5. The replacement removes complete hardware tiles.
6. LUT processing can run in parallel with active GEMM.
7. LUT processing does not extend the critical path.
8. Output reconstruction can be eliminated.
9. Estimated energy reduction is meaningful.
10. Expected benefit is larger than hardware integration complexity.

The customized hardware direction should be reconsidered if:

- Full-model MAC reduction remains below 3%
 - LUT storage reaches hundreds of megabytes without compression
 - Accuracy significantly degrades
 - LUT access becomes bandwidth-bound
 - Tile count does not decrease
 - The LUT path remains on the critical path
 - The energy benefit is marginal
 - A standard quantization or pruning method provides similar savings at lower cost
-

15. Recommended Next Experimental Matrix

The next software experiment should include the following configurations.

Configuration	Number of layers	Groups per layer	LUT precision
Conservative-1	4	8	FP16
Conservative-2	8	8	FP16
Balanced-1	8	12	FP16
Balanced-2	8	16	INT8
Aggressive-1	16	8	INT8
Aggressive-2	16	12	INT8
Adaptive	Per-layer	Per-layer selected	FP16/INT8

For every configuration, report:

- Total replaced MACs
- Full-model replaced MAC ratio
- Total LUT storage
- LUT reads per token
- Perplexity change
- Task-score change
- Long-generation stability
- Estimated cycle reduction
- Estimated energy reduction

The adaptive configuration should be treated as the primary deployment candidate.

16. Reporting Format for Management

Management should not receive a report centered on kernel-level details.

The report should answer four questions:

1. How much full-model computation can be eliminated?
2. What accuracy loss is introduced?
3. What hardware resources are required?
4. When can the method be tested on real customized hardware?

The main summary table should look like:

Configuration	Full-model MAC reduction	LUT storage	Task-quality change	Estimated hardware latency gain	Estimated energy gain
Conservative	3%	20 MB	Almost unchanged	Small	Small
Balanced	7%	50 MB	Minor degradation	Worth FPGA validation	Potentially meaningful
Aggressive	12%	100 MB	Noticeable degradation	High theoretical potential	Requires further tuning

The current message to management should be:

We have confirmed that LUT replacement can physically skip part of the dense matrix computation. The current GPU implementation does not reduce end-to-end latency because GPU execution introduces LUT and reconstruction overhead. The next milestone is to determine whether the method can achieve at least 5%–10% full-model computation reduction while maintaining acceptable model quality. If that target is achieved, the method should be evaluated on FPGA, NPU, CIM, or other customized hardware capable of running LUT lookup and matrix computation in parallel.

17. Hardware Resource Questions for Management

The team should ask management for specific information rather than generally asking when customized hardware will become available.

The following questions should be raised:

Existing Resources

- Does the company currently have FPGA development boards?
- Does the company have access to programmable NPU hardware?
- Is there an internal ASIC, CIM, or accelerator team?
- Does the company have a cycle-level hardware simulator?
- Is cloud FPGA access available?
- Are there existing partnerships with universities or chip companies?
- Is RTL engineering support available?
- Is compiler or kernel engineering support available?

Timeline

- Can FPGA access be arranged within the next three months?
- Is there a hardware prototype planned within the next six months?
- Is this method potentially relevant to the next hardware generation?
- When would the hardware team be available to review the dataflow?

- What evidence is required before hardware resources can be allocated?

Hardware Constraints

- What is the target SRAM capacity?
 - What LUT precision is supported?
 - What is the target array width?
 - What is the target memory bandwidth?
 - Is direct output placement supported?
 - Can GEMM and LUT operations run concurrently?
 - Is the deployment objective latency, throughput, energy, or cost?
 - What accuracy degradation is acceptable?
 - What minimum model-level computation reduction is considered worthwhile?
-

18. Suggested Management Communication

The immediate request to management should not be for a fully customized chip.

The request should be:

1. Confirm whether FPGA, NPU, CIM, or accelerator resources exist.
2. Confirm the expected availability timeline.
3. Confirm what full-model savings must be demonstrated before hardware support is provided.
4. Identify a hardware or RTL contact who can review the proposed dataflow.
5. Determine whether the team can access a simulator before physical hardware becomes available.

The recommended commitment from the software/model team is:

We will first complete multi-layer replacement, LUT quantization, full-model accuracy evaluation, and computation-reduction analysis. If the full-model reduction reaches at least 5%–10% with acceptable quality degradation, we will request FPGA or customized hardware validation.

19. Immediate Action Items

Model and Algorithm

- Select 8–16 candidate middle and late layers
- Test 4/8/12/16 groups per layer
- Build per-layer sensitivity profiles
- Train multi-layer replacement configurations
- Add FP16 LUT
- Add INT8 LUT
- Evaluate accumulated approximation error
- Run task-level benchmarks
- Test long-generation stability

Cost and Deployment

- Calculate per-layer eliminated MACs
- Calculate full-model eliminated MACs
- Calculate LUT reads per token
- Calculate total LUT storage
- Calculate LUT bandwidth
- Calculate dense weight traffic eliminated
- Compare fixed and adaptive configurations
- Build a full-model Pareto table

Hardware Preparation

- Define 64/128/256-column array models
- Calculate tile reduction
- Build a GEMM/LUT parallel cycle model
- Model direct output placement
- Model SRAM bandwidth
- Build an energy break-even model
- Prepare a one-page hardware resource request

Management Communication

- Report that physical computation skipping has been validated
- Explain that GPU is not the target deployment architecture
- Present the full-model validation plan
- Ask about FPGA/NPU/CIM access
- Ask about the expected hardware timeline
- Ask what metrics are required for hardware allocation
- Request a hardware-team contact

20. Final Direction

The current result should be treated as the completion of the single-layer proof-of-mechanism stage.

The next stage is not further GPU latency optimization.

The next stage is:

Single-layer physical skip
→ Multi-layer replacement
→ Full-model MAC reduction
→ LUT quantization
→ Real-task quality validation
→ Hardware cycle and energy modeling

- FPGA or programmable accelerator prototype
- Customized hardware decision

The project should continue only if multi-layer deployment produces meaningful full-model benefits.

The immediate success criterion is:

Achieve at least 5% full-model MAC reduction with acceptable task-quality degradation and manageable FP16 or INT8 LUT storage.

The stronger target is:

Achieve 10% or more full-model computation reduction while keeping the LUT path outside the hardware critical path.

Only after reaching this point should the team commit substantial resources to FPGA, CIM, ASIC, or other customized hardware implementations.